SRv6 实践与应用：IS-IS 打底、L3VPN 跑通、TE 选路及典型场景

朱古力

导语：在云网融合与算力调度的演进下，传统 MPLS 骨干网正面临协议栈臃肿的瓶颈。作为下一代 IP 统一方案，SRv6 的“源路由”理念赋予了网络前所未有的可编程能力。本文不谈空洞的协议白皮书，我们在真机（星融元 CX308P-48Y-M，AsterNOS）上从零搭建：以 IS-IS 打底构建无标签传送平面，通过 MP-BGP 贯通多租户 L3VPN，最终在真机抓包中剥离 SRH 扩展报头，剖析动静态选路的工程本质。

一、 SRv6 核心价值：智能与轻量

SRv6 的设计哲学可以总结为两个关键词：智能（可编程）与轻量（易部署）。

1. 智能：灵活的路由与业务调度

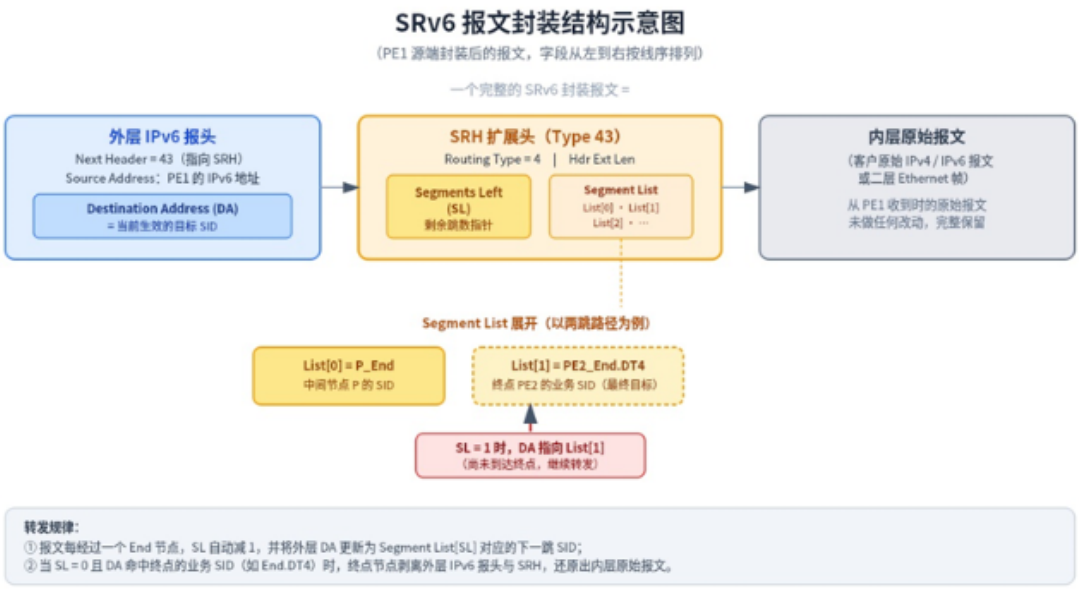
- 网络路径可编程：传统 IP 路由只能被动根据目的地址转发，而 SRv6 赋予了网络精细化调度能力，可通过灵活编排路径节点，实现特定业务流的按需引导。
- 无缝对接 SDN 架构：能完美对接上层控制器与业务系统，实现跨应用与网络传输层的联动。

2. 轻量：控制面简洁

- 极轻量控制面：基于现有 IGP 协议扩展（如 IS-IS / OSPFv3）即可分发信息，无需额外的标签分发或隧道信令协议，大幅降低控制面开销。
- 原生 IP 融合：完全能够运行于现有 IPv6 基础网络之上，真正实现“有 IPv6 路由的地方就能跑 SRv6”。

二、 核心数据结构：SRH 与 SID 解剖

在 SRv6 数据面中，有两个关键组件：SRH 扩展报头 与 SID（Segment Identifier）。



SRv6 报文封装结构：外层 IPv6 报头 + SRH 扩展头 + 内层原始报文

报文在源端被封装时，会插入类型为 43 的 SRH 扩展报头：

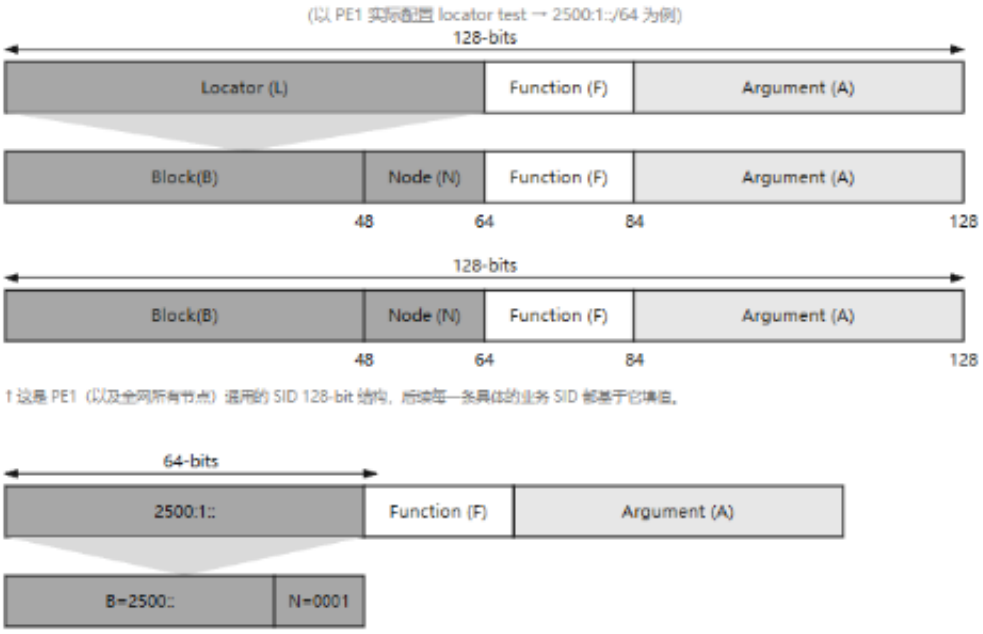
- Segment List（段列表）：记录数据包需要依次经过的节点路径（如 List[0]=PE2_SID, List[1]=P_SID）。
- Segments Left（SL，倒计时指针）：每经过一个节点，SL 自动减 1，引导报文转向下一个目标 SID。

2. SID 的 128-bit 语义拆解

每个 SID 本质上都是一个合法的 IPv6 地址，其内部结构划分为：

> SID = Locator（路由寻路）+ Function（指令动作）+ Arguments（附加参数）

这种设计让它既能当作普通 IPv6 地址查表转发，又能携带触发节点执行特定动作的指令。



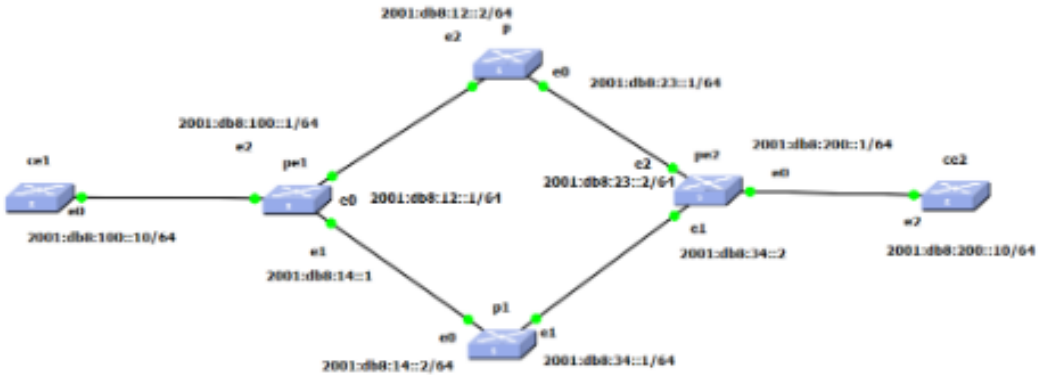
三、 核心指令行为（Behaviors）

当带有 SRH 的报文到达各节点时，节点会根据 SID 携带的 Function 执行对应的标准动作：

动作名称	行为说明
End SID	标准节点行为：处理 SRH，执行 SL--，将外层 IPv6 目的地址（DA）更新为下一个 SID，按 IPv6 路由转发。
End.X SID	邻居关联行为：处理 SRH 并更新 DA 后，强制从特定指定的出接口（End.X 绑定接口）转发出去。
End.DT4 SID	终点 IPv4 解封装：到达终点 PE 后，剥离外层 IPv6 报头与 SRH，露出内层 IPv4 报文，查 VPN 实例表转发。
End.DT6 SID	终点 IPv6 解封装：到达终点 PE 后，剥离外层 IPv6 报头与 SRH，露出内层 IPv6 报文，查 VPN 实例表转发。
End.DX2 SID	终点 L2VPN 解封装：到达终点 PE 后，剥离外层 IPv6 报头与 SRH，露出内层 二层 Ethernet 帧 并从绑定接口送出。

四、 经典拓扑落地

将上述理论映射到标准的骨干网拓扑中：



- PE1（源端）：负责将用户侧业务流量进行 SRv6 封装（注入 SRH 列表）。
- P、P1（中间节点）：利用标准的 End 行为进行纯 IPv6 转发与指针前移，无需维护繁重的标签交换表。
- PE2（终点）：通过 End.DT4 等终点行为剥离外层封装，将纯净的原始业务流送达 CE2。

五、 实验：从 IS-IS 到 L3VPN，再到 TE 选路

租户（VRF 100）的 ce1 访问 ce2。先让它走 p，再让 pe1 通过 SID 列表指定“经 p”或“经 p1”，并在抓包里看到 SRH。

设备是星融元 CX308P-48Y-M（AsterNOS，SONiC.Campus-V6.1R007T004），本文所有配置命令里的接口号都是设备里的编号；拓扑图和抓包窗口标题用的是 GNS3 编号，对着图配置时先把图上的编号加 1。完整配置见文末附录。

1. 打底：用 IS-IS 动态路由打通底层，BGP 跑通 L3VPN

思路很直接：四台设备（PE1、PE2、P、P1）跑同一个 IS-IS 进程，把直连网段、Loopback 和 Locator 全部重分发进去，连通性完全由 IS-IS 提供。

```
sonic(config)# router isis test
sonic(config-router)# net <本机 NET>
sonic(config-router)# is-type level-1
sonic(config-router)# redistribute ipv6 connected level-1
```

除了启用 IS-IS，还有两条值得说一下（以 PE1 的 Ethernet 1 为例）：

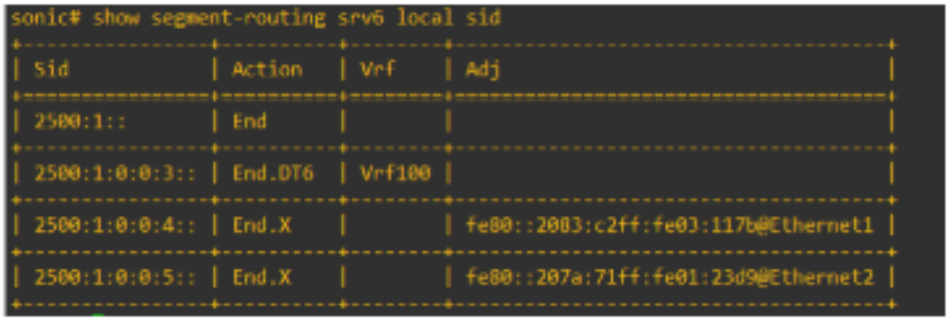
```
sonic(config)# interface ethernet 1
sonic(config-if)# ipv6 router isis test
sonic(config-if)# no isis hello padding
sonic(config-if)# ipv6 use-link-local
```

```
sonic(config-if)# shutdown
sonic(config-if)# no shutdown
```

值得注意的有两点：

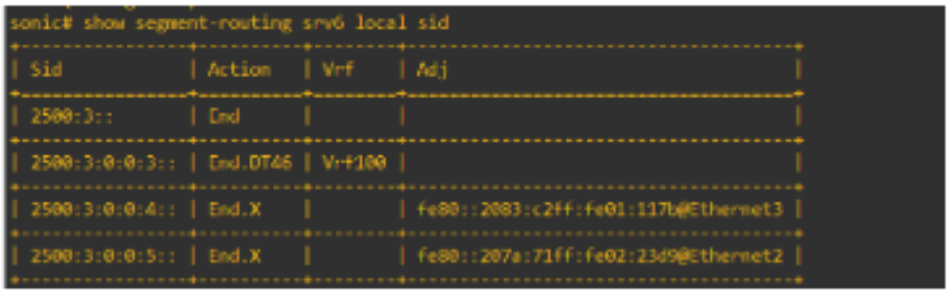
- 接口 MTU 是 9216，需要关掉 hello padding，否则因小包填充或 MTU 协商不匹配导致邻居无法卡死在 Init 状态。
- 核心口下要配 ipv6 use-link-local。在配置完所有路由后一定要统一下 shutdown / no shutdown 触发邻居重建。

用 show segment-routing srv6 local sid 查看。本实验中 PE1 上的 VPN SID 显示为 End.DT6，PE2 上显示为 End.DT46，行为没有差异，但都能完成 IPv6 VPN 的解封装转发。



PE1 的 local sid 输出

2500:1:0:0:4::绑定到 Ethernet1（p），2500:1:0:0:5::绑定到 Ethernet1（p1），2500:1:0:0:3::是进入 Vrf100 的 End.DT6。



PE2 的 local sid 输出

2500:3:0:0:5::绑定到 Ethernet2（p1），2500:3:0:0:4::绑定到 Ethernet3（p），2500:3:0:0:3::是进入 Vrf100 的 End.DT6。

再用 BGP 把租户业务送过去。

PE1、PE2 之间建立 MP-BGP IPv6 VPN 邻居，VRF 100 下导出 VPN SID；PE 与 CE 之间跑 EBGP（以 PE1 为例）：

```
sonic(config)# router bgp 65000
sonic(config-router)# neighbor 2000::3 remote-as 65000
sonic(config-router)# neighbor 2000::3 update-source 2000::1
sonic(config-router)# neighbor 2000::3 capability extended-nexthop
sonic(config-router)# address-family ipv6 vpn
sonic(config-router-af)# neighbor 2000::3 activate
sonic(config)# router bgp 65000 vrf 100
sonic(config-router)# neighbor 2001:db8:100::10 remote-as 65001
sonic(config-router)# address-family ipv6 unicast
sonic(config-router-af)# neighbor 2001:db8:100::10 activate
```

```
sonic(config-router-af)# export vpn
sonic(config-router-af)# import vpn
sonic(config-router-af)# rd vpn export 1:1
sonic(config-router-af)# redistribute connected
sonic(config-router-af)# rt vpn both 100:100
sonic(config-router-af)# sid vpn export auto
```

PE2 对称配置：iBGP 邻居换成 2000::1，update-source 为 2000::3；VRF 里的 EBGP 邻居是 CE2 的 2001:db8:200::10（AS 65002），rd 用 2:2。

CE 侧也要配 BGP：CE 和自己直连的 PE 建 EBGP，并把自己的网段发给 PE（以 CE1 为例）：

```
sonic(config)# router bgp 65001
sonic(config-router)# no bgp default ipv4-unicast
sonic(config-router)# no bgp ebgp-requires-policy
sonic(config-router)# neighbor 2001:db8:100::1 remote-as 65000
sonic(config-router)# address-family ipv6 unicast
sonic(config-router-af)# neighbor 2001:db8:100::1 activate
sonic(config-router-af)# redistribute connected
```

CE2 同理，详细可见附件指令集。

2. 架构师视角：动态 Metric 重算 vs 静态 SRv6 TE 编排

在生产环境中，当遇到“业务流量需要绕路”的需求时，工程师的第一反应往往是调整 IS-IS 的 Metric。但在真机验证后，我们必须理清两者的工程边界：

1. 精准控制（静态 SRv6 路由）：

- 本质：源端 PE 显式封装 Segment List。
- 优势：在缺乏集中式 SDN 控制器的轻量化场景下，通过在源端 PE 编写静态路由绑定特定 Segment List，可以精准实现“按需点名、去回程解耦、服务链引流”。抓包可见清晰的 Type 43 SRH 扩展头。

TE 还需要一个物理口做 SRv6 环回，要用没接线的口（本例 ethernet 4），而且顺序不能反：

```
sonic(config)# interface ethernet 4
sonic(config-if)# loopback internal
sonic(config-if)# loopback binding srv6
```

切换到 P：

PE1：

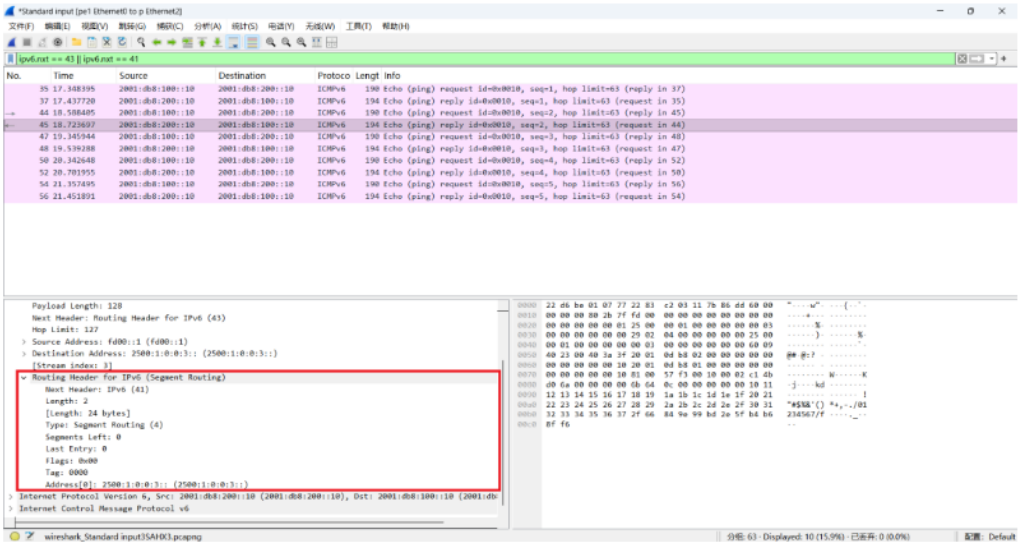
```
sonic(config)# vrf 100
sonic(config-vrf)# no ipv6 route 2001:db8:200::/64 2001:db8:14::2 ne…
sonic(config-vrf)# ipv6 route 2001:db8:200::/64 2001:db8:12::2 ne…
```

注：原文代码块较宽，截图中行尾被横向滚动条截断，此处按截图可见内容录入，省略号处为未显示部分。

PE2：

```
sonic(config)# vrf 100
sonic(config-vrf)# no ipv6 route 2001:db8:100::/64 2001:db8:34::1 ne…
sonic(config-vrf)# ipv6 route 2001:db8:100::/64 2001:db8:23::1 ne…
```

下面是业务经 P 的状态：P1 一侧的链路是空的，P 一侧能看到 ICMPv6 往返。





切换到 P1：

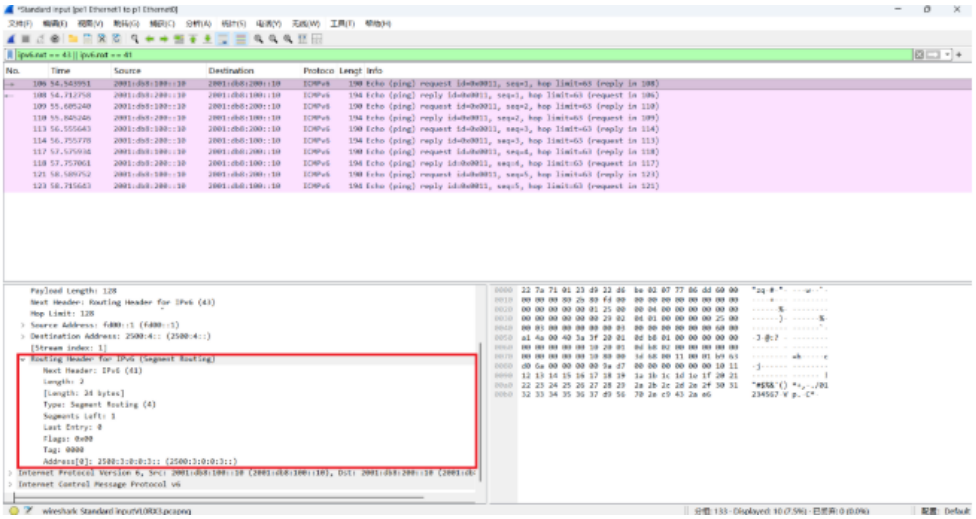
PE1：

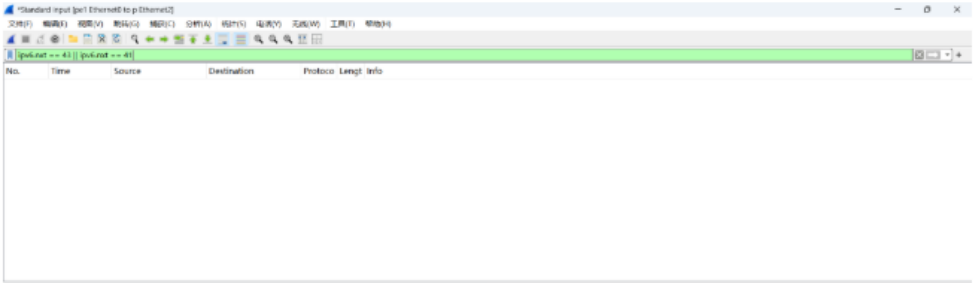
```
sonic(config)# vrf 100
sonic(config-vrf)# no ipv6 route 2001:db8:200::/64 2001:db8:12::2 ne…
sonic(config-vrf)# ipv6 route 2001:db8:200::/64 2001:db8:14::2 ne…
```

PE2：

```
sonic(config)# vrf 100
sonic(config-vrf)# no ipv6 route 2001:db8:100::/64 2001:db8:23::1 ne…
sonic(config-vrf)# ipv6 route 2001:db8:100::/64 2001:db8:34::1 ne…
```

而下面是业务经 P1 的状态：P 一侧的链路是空的，P1 一侧能看到 ICMPv6 往返。





其中红框是 SRH（Segment Routing Header，路由类型 4 的 IPv6 扩展头）

2. 动态调优（改 IS-IS Metric）：

- 本质：触发 IGP 重新计算最短路径树（SPF）。
- 痛点：它是一把“钝刀子”——整张网、所有同类业务会跟着一起切；且报文中没有 Segment List，Next Header 仍为 41，你无法在数据面上“指名道姓”让流量走特定节点。

切到 P：

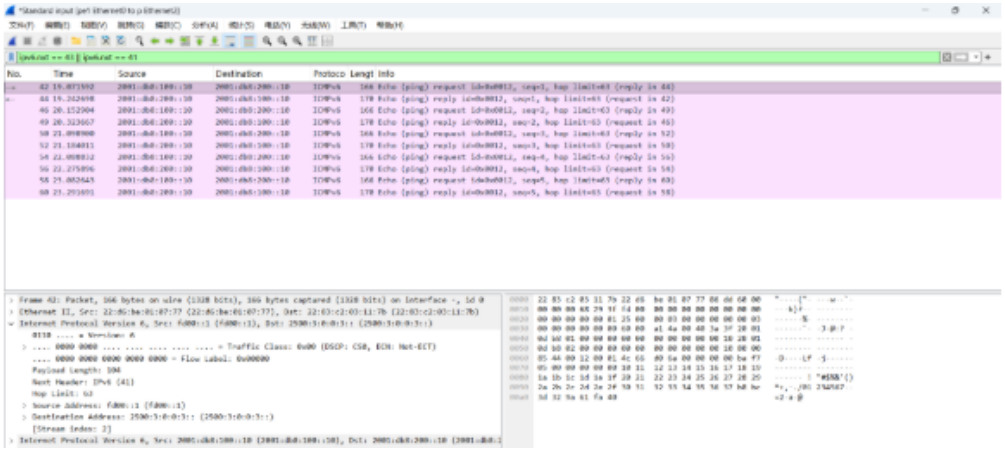
PE1：

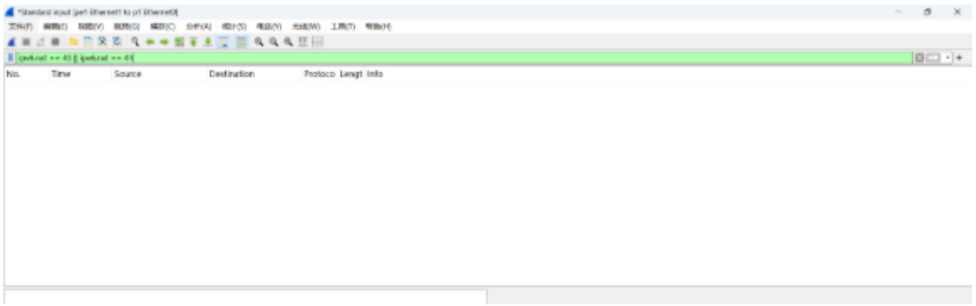
```
sonic(config)# interface ethernet 1
sonic(config-if)# no isis metric
sonic(config)# interface ethernet 2
sonic(config-if)# isis metric 100
```

PE2：

```
sonic(config)# interface ethernet 3
sonic(config-if)# no isis metric
sonic(config)# interface ethernet 2
sonic(config-if)# isis metric 100
```

下面是业务经 P 的状态：P1 一侧的链路是空的，P 一侧能看到 ICMPv6 往返。





切换到 P1：

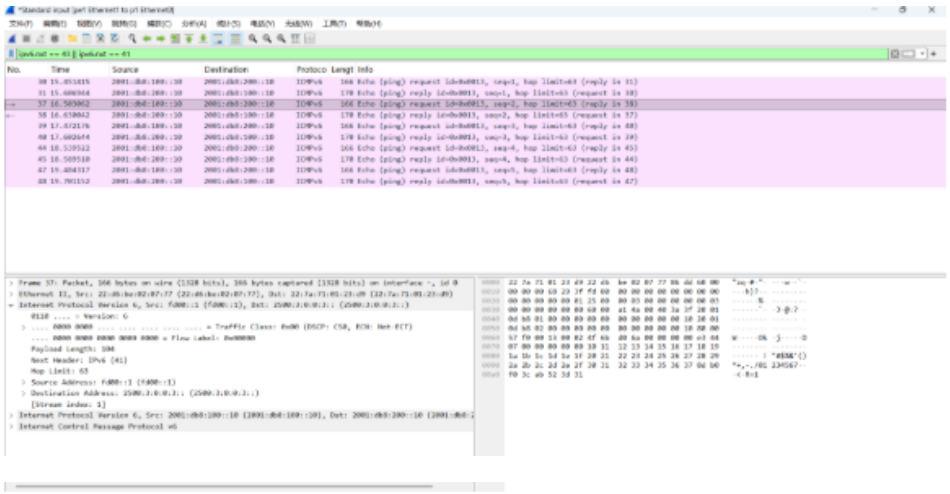
PE1：

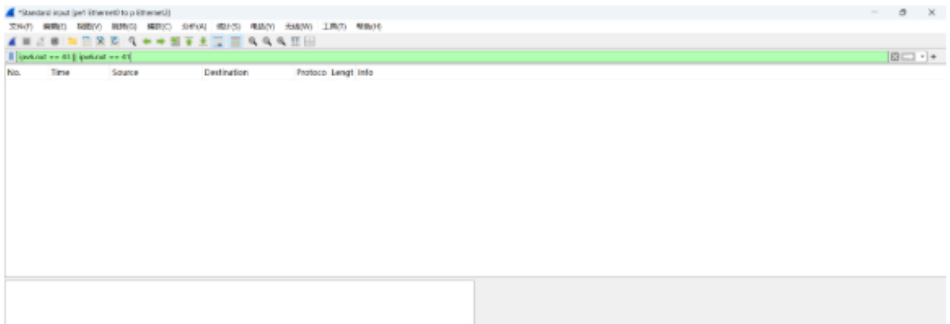
```
sonic(config)# interface ethernet 2
sonic(config-if)# no isis metric
sonic(config)# interface ethernet 1
sonic(config-if)# isis metric 100
```

PE2：

```
sonic(config)# interface ethernet 2
sonic(config-if)# no isis metric
sonic(config)# interface ethernet 3
sonic(config-if)# isis metric 100
```

而下面是业务经 P1 的状态：P 一侧的链路是空的，P1 一侧能看到 ICMPv6 往返。





值得注意的是点开一个包，没有 SRH。动态方式确实能换路，改一个参数，全网路径跟着变。但仔细看这几张图：报文里始终只有一个目的 PE 的 VPN SID，没有 Segment List。路径是 IS-IS 按 metric 算出来的，我们只能影响它，没法在报文里写“经过谁”。

3. 动态 Metric 重算 vs 静态 SRv6 TE 路径编排特性对比

	动态：改 IS-IS metric	静态：SRv6 路由（Segment List）
谁决定路径	IS-IS 按 metric 计算，我们只能间接影响	管理员在 PE 上明确指定
控制粒度	整张网、所有业务一起切	精确到某个目的前缀
Segment List	没有，报文里只有目的 PE 的 VPN SID	封装在 SRH 里，途经节点逐个写明
报文形态	无 SRH，Next Header 41	带 SRH，Next Header 43
去程 / 回程	两端都要改，漏改一端就不一致	去程、回程各自明确指定
核验方式	看报文出现在哪条链路	看链路，再看 SRH 里的 SID 和 Segments Left
回退	把 metric 改回去	删掉这一条路由

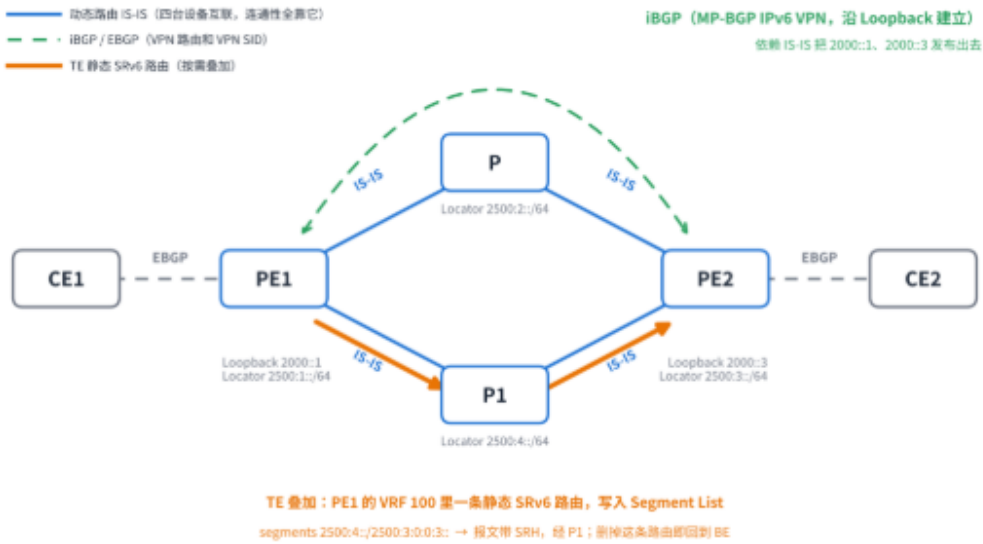
结论很直接：改 metric 只能让“整张网”换一条最短路，路径是算出来的，报文里没有 Segment List，也区分不了业务。而 SRv6 TE 要做的恰好是“让某个业务必须走指定路径”，这要求在源端 PE 上为这个业务明确写出封装的 Segment List。IS-IS 不会产生它，所以在没有控制器的环境里，只能靠静态 SRv6 路由来指定业务的转发路径和 Segment List。

工程选型建议：应对大面积网络检修（如整条骨干链路维护），改 Metric 最高效；面对精细化 SLA 保障或多路径流量调度，静态 SRv6 路由才是正解。

六、 典型应用场景：SRv6 能怎么用

做完这组实验，我最大的感受不是“路径能指定”，而是“不用一步到位”。

回头看整个过程，我其实是这样一步步搭起来的。



场景一：广域网与运营商骨干网的 VPN 业务

骨干网承载租户 VPN 互联时，传统做法需部署 LDP、RSVP-TE 等繁杂协议。通过 SRv6 改造，P 和 P1 上只需运行 IS-IS 和 SRv6 Locator，无需维护 MPLS 标签、VRF 或 BGP。默认 BE 模式下报文走最短路径；当需要指定路径时，在头节点 PE 按目的前缀写入静态 SRv6 路由即可，去回程可独立解耦。

场景二：数据中心互联，还能“点名经过某个节点”

数据中心互联时，可将 DC 出口作为 PE 部署。借助 Segment List 指定经由特定的 P 或 P1 节点转发，满足特殊的安全合规或服务链（Service Chaining）引流需求。

场景三：多路径调度，能切，也能同时走

当存在多条可选路径时，Metric 相同时默认走 ECMP 负载分担；若配置了多条静态 TE 路由，同样可实现多路径同时分流或精准主备切换。

场景四：核心链路检修，先把业务引开

核心 P 节点需要检修时，可通过在两端 PE 同步调大对应接口的 IS-IS Metric，将流量无损引流至 P1 侧。切流时必须保证双向对称，避免去回程不一致。

场景五：平滑演进，一层层往上叠

在现网引入 SRv6 建议采取渐进式策略：核心设备先配置 Locator 并通过 IS-IS 发布 ---> 先用 BE 把业务跑通 ---> 需要换路时改 Metric ---> 最后根据 SLA 需求叠加 TE 策略。每一次叠加只需动两端 PE，中间设备和 VRF 零修改，做到了真正的平滑演进。

七、 总结与工程感悟

这次实验做下来，最大的收获不是哪条命令，而是几个以前没这么清楚的认识。

1. 先通，再谈优 复杂的网络改造永远不能“一口吃成胖子”。从 IS-IS 底座 ---> 发布 ---> iBGP 邻居建立 ---> VPN 贯通，每一层必须正向可验证。顺序颠倒，排错时就会陷入盲目敲 debug 的泥潭。

2. 路径要“可编程”，手里得有“手术刀” 改 IS-IS Metric 是一把整网切流的“钝刀子”，而静态 SRv6 路由则是精细化调度的“手术刀”。在缺乏控制器的轻量化场景下，理解如何在源端 PE 编写 Segment List，是网络工程师掌控数据面的分水岭。

3. 命令是表象，报文是真相 邻居起不来是因为 hello padding，路由处于 Inactive 是因为 nexthop-vrf 没对齐。理解报文里每一字节的变动（如 SRH 扩展头与 SL 倒计时指针），远比死记硬背厂商 CLI 命令更有价值。

4. 永远给自己留好“回退方案” 每一次叠加配置前，都要想清楚怎么撤回（如 no isis metric 或删除重写 TE 路由）。有退路，才敢在生产环境中大胆演进。

不要沦为单纯记忆厂商命令行的工具人。理解报文怎么走，才是网络工程师的核心护城河。

附录：完整指令集

P：

```
sonic# configure
sonic(config)# router isis test
sonic(config-router)# net 10.0000.0000.0000.0002.00
sonic(config-router)# is-type level-1
sonic(config-router)# redistribute ipv6 connected level-1
sonic(config-router)# exit
sonic(config)# interface ethernet 3
sonic(config-if)# ip address 2001:db8:12::2/64
sonic(config-if)# ipv6 router isis test
sonic(config-if)# no isis hello padding
sonic(config-if)# ipv6 use-link-local
sonic(config-if)# exit
sonic(config)# interface ethernet 1
sonic(config-if)# ip address 2001:db8:23::1/64
sonic(config-if)# ipv6 router isis test
sonic(config-if)# no isis hello padding
sonic(config-if)# ipv6 use-link-local
sonic(config-if)# exit
sonic(config)# segment-routing
sonic(config-sr)# srv6
sonic(config-srv6)# locators
sonic(config-srv6-locators)# locator LOC-P
sonic(config-srv6-locator)# prefix 2500:2::/64
sonic(config-srv6-locator)# exit
sonic(config-srv6-locators)# exit
sonic(config-srv6)# exit
sonic(config-sr)# exit
sonic(config)# router isis test
sonic(config-router)# segment-routing srv6
sonic(config-router-srv6)# locator LOC-P
sonic(config-router-srv6)# exit
sonic(config-router)# exit
sonic(config)# end
```

P1：

```
sonic# configure
sonic(config)# router isis test
sonic(config-router)# net 10.0000.0000.0000.0004.00
sonic(config-router)# is-type level-1
sonic(config-router)# redistribute ipv6 connected level-1
sonic(config-router)# exit
sonic(config)# interface ethernet 1
sonic(config-if)# ip address 2001:db8:14::2/64
sonic(config-if)# ipv6 router isis test
sonic(config-if)# no isis hello padding
sonic(config-if)# ipv6 use-link-local
sonic(config-if)# exit
sonic(config)# interface ethernet 2
sonic(config-if)# ip address 2001:db8:34::1/64
sonic(config-if)# ipv6 router isis test
sonic(config-if)# no isis hello padding
sonic(config-if)# ipv6 use-link-local
sonic(config-if)# exit
sonic(config)# segment-routing
sonic(config-sr)# srv6
```

```
sonic(config-srv6)# locators
sonic(config-srv6-locators)# locator LOC-P1
sonic(config-srv6-locator)# prefix 2500:4::/64
sonic(config-srv6-locator)# exit
sonic(config-srv6-locators)# exit
sonic(config-srv6)# exit
sonic(config-sr)# exit
sonic(config)# router isis test
sonic(config-router)# segment-routing srv6
sonic(config-router-srv6)# locator LOC-P1
sonic(config-router-srv6)# exit
sonic(config-router)# exit
sonic(config)# end
```

PE1：

```
sonic# configure
sonic(config)# router isis test
sonic(config-router)# net 10.0000.0000.0000.0001.00
sonic(config-router)# is-type level-1
sonic(config-router)# redistribute ipv6 connected level-1
sonic(config-router)# exit
sonic(config)# interface ethernet 1
sonic(config-if)# ip address 2001:db8:12::1/64
sonic(config-if)# ipv6 router isis test
sonic(config-if)# no isis hello padding
sonic(config-if)# ipv6 use-link-local
sonic(config-if)# exit
sonic(config)# interface ethernet 2
sonic(config-if)# ip address 2001:db8:14::1/64
sonic(config-if)# ipv6 router isis test
sonic(config-if)# no isis hello padding
sonic(config-if)# ipv6 use-link-local
sonic(config-if)# exit
sonic(config)# vrf 100
sonic(config-vrf)# exit
sonic(config)# interface ethernet 3
sonic(config-if)# vrf 100
sonic(config-if)# ip address 2001:db8:100::1/64
sonic(config-if)# exit
sonic(config)# interface loopback 1
sonic(config-if)# ip address 2000::1/128
sonic(config-if)# ipv6 router isis test
sonic(config-if)# exit
sonic(config)# segment-routing
sonic(config-sr)# srv6
sonic(config-srv6)# locators
sonic(config-srv6-locators)# locator LOC-PE1
sonic(config-srv6-locator)# prefix 2500:1::/64
sonic(config-srv6-locator)# exit
sonic(config-srv6-locators)# exit
sonic(config-srv6)# exit
sonic(config-sr)# exit
sonic(config)# router isis test
sonic(config-router)# segment-routing srv6
sonic(config-router-srv6)# locator LOC-PE1
sonic(config-router-srv6)# exit
sonic(config-router)# exit
sonic(config)# router bgp 65000
sonic(config-router)# bgp router-id 1.1.1.1
sonic(config-router)# no bgp default ipv4-unicast
```

```
sonic(config-router)# no bgp ebgp-requires-policy
sonic(config-router)# segment-routing srv6
sonic(config-router-srv6)# locator LOC-PE1
sonic(config-router-srv6)# exit
sonic(config-router)# neighbor 2000::3 remote-as 65000
sonic(config-router)# neighbor 2000::3 capability extended-nexthop
sonic(config-router)# neighbor 2000::3 update-source 2000::1
sonic(config-router)# address-family ipv6 vpn
sonic(config-router-af)# neighbor 2000::3 activate
sonic(config-router-af)# exit
sonic(config-router)# exit
sonic(config)# router bgp 65000 vrf 100
sonic(config-router)# bgp router-id 1.1.1.1
sonic(config-router)# no bgp ebgp-requires-policy
sonic(config-router)# neighbor 2001:db8:100::10 remote-as 65001
sonic(config-router)# address-family ipv6 unicast
sonic(config-router-af)# neighbor 2001:db8:100::10 activate
sonic(config-router-af)# export vpn
sonic(config-router-af)# import vpn
sonic(config-router-af)# rd vpn export 1:1
sonic(config-router-af)# redistribute connected
sonic(config-router-af)# rt vpn both 100:100
sonic(config-router-af)# sid vpn export auto
sonic(config-router-af)# exit
sonic(config-router)# exit
sonic(config)# end
```

PE2：

```
sonic# configure
sonic(config)# router isis test
sonic(config-router)# net 10.0000.0000.0000.0003.00
sonic(config-router)# is-type level-1
sonic(config-router)# redistribute ipv6 connected level-1
sonic(config-router)# exit
sonic(config)# interface ethernet 3
sonic(config-if)# ip address 2001:db8:23::2/64
sonic(config-if)# ipv6 router isis test
sonic(config-if)# no isis hello padding
sonic(config-if)# ipv6 use-link-local
sonic(config-if)# exit
sonic(config)# interface ethernet 2
sonic(config-if)# ip address 2001:db8:34::2/64
sonic(config-if)# ipv6 router isis test
sonic(config-if)# no isis hello padding
sonic(config-if)# ipv6 use-link-local
sonic(config-if)# exit
sonic(config)# vrf 100
sonic(config-vrf)# exit
sonic(config)# interface ethernet 1
sonic(config-if)# vrf 100
sonic(config-if)# ip address 2001:db8:200::1/64
sonic(config-if)# exit
sonic(config)# interface loopback 1
sonic(config-if)# ip address 2000::3/128
sonic(config-if)# ipv6 router isis test
sonic(config-if)# exit
sonic(config)# segment-routing
sonic(config-sr)# srv6
sonic(config-srv6)# locators
sonic(config-srv6-locators)# locator LOC-PE2
```

```
sonic(config-srv6-locator)# prefix 2500:3::/64
sonic(config-srv6-locator)# exit
sonic(config-srv6-locators)# exit
sonic(config-srv6)# exit
sonic(config-sr)# exit
sonic(config)# router isis test
sonic(config-router)# segment-routing srv6
sonic(config-router-srv6)# locator LOC-PE2
sonic(config-router-srv6)# exit
sonic(config-router)# exit
sonic(config)# router bgp 65000
sonic(config-router)# bgp router-id 2.2.2.2
sonic(config-router)# no bgp default ipv4-unicast
sonic(config-router)# no bgp ebgp-requires-policy
sonic(config-router)# segment-routing srv6
sonic(config-router-srv6)# locator LOC-PE2
sonic(config-router-srv6)# exit
sonic(config-router)# neighbor 2000::1 remote-as 65000
sonic(config-router)# neighbor 2000::1 capability extended-nexthop
sonic(config-router)# neighbor 2000::1 update-source 2000::3
sonic(config-router)# address-family ipv6 vpn
sonic(config-router-af)# neighbor 2000::1 activate
sonic(config-router-af)# exit
sonic(config-router)# exit
sonic(config)# router bgp 65000 vrf 100
sonic(config-router)# bgp router-id 2.2.2.2
sonic(config-router)# no bgp ebgp-requires-policy
sonic(config-router)# neighbor 2001:db8:200::10 remote-as 65002
sonic(config-router)# address-family ipv6 unicast
sonic(config-router-af)# neighbor 2001:db8:200::10 activate
sonic(config-router-af)# export vpn
sonic(config-router-af)# import vpn
sonic(config-router-af)# rd vpn export 2:2
sonic(config-router-af)# redistribute connected
sonic(config-router-af)# rt vpn both 100:100
sonic(config-router-af)# sid vpn export auto
sonic(config-router-af)# exit
sonic(config-router)# exit
sonic(config)# end
```

CE1：

```
sonic# configure
sonic(config)# interface ethernet 1
sonic(config-if)# ip address 2001:db8:100::10/64
sonic(config-if)# exit
sonic(config)# router bgp 65001
sonic(config-router)# no bgp default ipv4-unicast
sonic(config-router)# no bgp ebgp-requires-policy
sonic(config-router)# neighbor 2001:db8:100::1 remote-as 65000
sonic(config-router)# address-family ipv6 unicast
sonic(config-router-af)# neighbor 2001:db8:100::1 activate
sonic(config-router-af)# redistribute connected
sonic(config-router-af)# exit
sonic(config-router)# exit
sonic(config)# end
```

CE2：

```
sonic# configure
sonic(config)# interface ethernet 3
sonic(config-if)# ip address 2001:db8:200::10/64
sonic(config-if)# exit
sonic(config)# router bgp 65002
sonic(config-router)# no bgp ebgp-requires-policy
sonic(config-router)# no bgp default ipv4-unicast
sonic(config-router)# neighbor 2001:db8:200::1 remote-as 65000
sonic(config-router)# address-family ipv6 unicast
sonic(config-router-af)# neighbor 2001:db8:200::1 activate
sonic(config-router-af)# redistribute connected
sonic(config-router-af)# exit
sonic(config-router)# exit
sonic(config)# end
```